



A metaheuristic algorithm for efficient aircraft sequencing and scheduling in terminal maneuvering areas

Dongdong Gui¹ · Meilong Le¹ · Xiaodong Luo³ · Zhouchun Huang² 

Received: 5 April 2024 / Accepted: 13 September 2024 / Published online: 23 September 2024
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract

Air traffic controllers face a daily challenge ensuring the safe and orderly flow of aircraft in terminal maneuvering areas, known academically as the Aircraft Sequencing and Scheduling Problem (ASSP). Typically formulated as a mixed integer linear programming problem, the strong NP-hard nature of the ASSP necessitates the use of heuristics or metaheuristics for practical, time-efficient solutions. In this paper, we propose a new metaheuristic algorithm to address the computational complexity of the problem. The algorithm employs a single-solution-based search methodology that starts from a well-constructed initial solution and iteratively explores a fix-and-optimize procedure to find an improved solution. Specifically, this procedure solves a variant of the ASSP that contains all the continuous variables and a selected subset of binary variables of the original problem, with the remaining binary variables being temporarily fixed to the values determined based on previous iterations. Additionally, the algorithm incorporates tabu search to increase the possibility of quickly finding optimal solutions. A rolling horizon control approach further accelerates the solution derivation process by decomposing the original problem into smaller sub-problems. Our computational experiments using real traffic data from Guangzhou Baiyun International Airport demonstrates the effectiveness of the algorithm in terms of solution quality and computational time.

Keywords Aircraft sequencing and scheduling · Metaheuristic algorithm · Mixed integer linear programming · Fix-and-optimize

1 Introduction

Air transport demand has seen pronounced growth in recent decades. Despite the significant impact of the COVID-19 outbreak on air transportation, a recent comprehensive assessment of the global traffic situation indicates that 94.2% of air traffic has now been restored compared to the level in 2019 [1]. Rising demand for air transport is expected to intensify congestion and delays in airports and the

surrounding airspace. This surge presents a significant challenge for Air Traffic Controllers (ATCs), particularly within the Terminal Maneuvering Areas (TMAs), where they are mandated to manage an ever-increasing volume of air traffic in a constrained airspace. The TMA is an airspace area surrounding one or more neighboring airports, designed explicitly to manage aircraft arrivals and departures. ATCs are tasked with the responsibility of maintaining a safe and orderly flow of aircraft in the TMA. This challenge is known as the Aircraft Sequencing and Scheduling Problem (ASSP). As a crucial component of air traffic management, the ASSP aims to enhance the utilization of TMA resources and reduce congestion [2, 3]. However, the ASSP exemplifies a classic NP-hard problem, implying it is unlikely to find efficient algorithms for its solution [4, 5].

The inherent complexity of the ASSP arises from the demand to yield optimal solutions given the varying safe separation requirements among various aircraft types. An exponential increase in computational demands is observed as the size of the problem extends. Exact solution methods like mixed integer programming or integer programming are frequently implemented to acquire optimal solutions. Nonetheless, the solution time also rises as the number of decision variables and constraints within the problem expands. Given this context, the application of heuristics or metaheuristics, which can generate near-optimal solutions within a short time, is necessitated.

In this paper, we propose a new metaheuristic algorithm for the ASSP, with the aim of significantly enhancing the quality of solutions in terms of both the objective function value and computation time. Our studied ASSP involves the simultaneous determination of the sequence of aircraft at each waypoint, as well as the scheduled time of aircraft arrival at these waypoints. These waypoints may include several runways and air waypoints. The objective function of the ASSP is to minimize a linear combination of total flight delay time and total dwell time. We use a mixed integer linear programming model, represented by a directed graph, to formulate the ASSP.

1.1 Literature review

As there exist comprehensive reviews in [6–9] and [10], our literature review will concentrate on two primary research streams: (i) the development of microscopic models for the ASSP and (ii) the development of solution methods for solving the models in this first category.

Pertaining to research stream (i), ASSP models can be classified as basic or detailed. Basic models only concentrate on the runways, whereas detailed models present a more comprehensive perspective incorporating not only the runways but also a variety of TMA resources. Beasley et al. [11] propose the most widely cited basic ASSP mathematical formulation, drawing parallels with a job-shop scheduling problem. The runways are interpreted as the machines, and the aircraft to be sequenced are the jobs. For a more realistic context, researchers extend their focus

to various TMA resources impacting aircraft landings and take-offs. Bianco et al. [12] present the detailed ASSP model based on the alternative graph formulation of [13] by taking into account the restrictions of various TMA resources for aircraft landings and take-offs, such as holding circles, time windows for aircraft travel times, multiple capacities of air segments and blocking constraints at runways. This work describes the relation between detailed ASSP and no-wait job shop scheduling problem. Here, the runways and air segments are considered as machines, and the aircraft to be sequenced and scheduled are the jobs. The separation time between each pair of aircraft corresponds to the sequence-dependent set-up time. The earliest and latest travel times are interpreted as the release date and deadline, respectively. The target landing (or take-off) time corresponds to the due date. Various studies, such as [14, 15] and [16–19], have successively extended the detailed ASSP model incorporating more assumptions and objectives, thereby enhancing the level of accuracy in modeling the ASSP. Our previous work [20] presents a detailed ASSP model using a directed graph. Building on this, the current study likens a detailed ASSP to a job shop scheduling problem and employs the same directed graph formulation for its modeling.

With respect to research stream (ii), various metaheuristic and exact algorithms are proposed to solve the ASSP. Within exact algorithms literature, Balakrishnan and Chandran [4] amalgamate the technique of constrained position shifting with a dynamic programming approach to solve a basic ASSP. In comparison, Faye [21] presents a dynamic constraint generation algorithm for the basic ASSP. Furthermore, D'Ariano et al. [15] propose a branch and bound algorithm specifically developed to address a detailed ASSP with fixed routes. However, exact algorithms are only capable of quickly computing near-optimal solutions for quite small instances or simplified problems. Therefore, the surge in proposals of numerous metaheuristics demonstrate an attempt to find good quality solutions within a short computation time. The commonly used metaheuristic algorithms include the genetic algorithm [22–24], the scatter search [25], the bionomic algorithm [25], the tabu search [18, 26, 27], the ant colony optimization [28, 29] and the simulated annealing [30, 31]. Moreover, Salehipour et al. [32] develop a Variable Neighborhood Descent (VND) to address the ASSP, wherein four neighborhood structures were designed to improve the incumbent solutions. Salehipour et al. [33] consider the same problem as [32] and use the VND in a hybrid algorithm by applying the simulated annealing framework. Sabar et al. [34] forward an iterated local search to solve the ASSP, using four perturbation operators and a VND to enable efficient solution space exploration. Recently, Ahmadian and Salehipour [35] introduce a novel metaheuristic algorithm for solving the aircraft landing problem (the basic ASSP). The algorithm can be decomposed into two steps. Firstly, an initial sequence of aircraft landing at the runway is generated using the target landing time first rule. Then, the sequence is improved by a “Relax-and-Solve” procedure. They called it “Relax-and-Solve” because it improves the aircraft sequence at the runway by iteratively deconstructing (relaxing) a sub-sequence of the current sequence and reconstructing (solving) a feasible sequence using

optimization techniques. With the sequence fixed, the runway scheduling problem is formed as a linear program, and the aircraft landing time is obtained by solving the linear program with the solver CPLEX. The difference between our study and [35] lies in the investigated problem. Specifically, we consider a detailed ASSP model rather than the classic aircraft landing problem. As a result, our sequence encoding, initial solution generation, and sequence improvement methods within our algorithm differ significantly from those proposed by [35].

1.2 Our contributions

The primary contribution of our study lies in the development of a novel solution algorithm for the ASSP, namely the Iterative Fix-and-Optimize (IFO) algorithm. The algorithm involves decomposing the ASSP into a chain of “easier-to-solve” subproblems with all the continuous variables and a selected subset of binary variables. The remaining binary variables are temporarily fixed to values determined from previous iterations. It further incorporates tabu search within the search phase to efficiently explore the solution space and employs a Rolling Horizon Control (RHC) approach to expedite the solution derivation process. We conduct extensive experiments using real traffic data from Guangzhou Baiyun International Airport (GBIA), and the computational results demonstrate the effectiveness and efficiency of our proposed solution approach.

1.3 Organisation of the paper

The organisation of the remainder of this paper is summarised as follows: Sect. 2 provides a description to the ASSP and formulates the problem as a mathematical program. Section 3 presents a solution method for addressing the ASSP. In Sect. 4, numerical experiments are conducted and the corresponding computational results are presented. Finally, Sect. 5 offers a conclusion that summarizes the findings of this study and suggests potential directions for future research.

2 Problem description and formulation

2.1 Problem description

In a TMA, arriving aircraft aim to navigate from their entry waypoint to a runway via prescribed landing air routes, following a standard descent profile. Departing aircraft have a similar goal, leaving the runway and flying towards the exit waypoint of the TMA via pre-defined take-off air routes, all while adhering to a standard ascent profile. All the aircraft must persistently maintain a minimum safety distance (safety separation) from each other in order to avoid the wake vortex effect or collision, depending

on the aircraft type and position. The distance-based separation can be converted into a time-based separation by taking into account the different aircraft speeds.

Each aircraft, whether arriving or departing, is obligated to reach its entry waypoint (or leave the runway) within a specific window between its earliest and latest allowed arrival or departure times. Then the landing or take-off procedure can commence based on the aircraft's current position and speed. Each aircraft possesses a travel time for flying through the air segments, which is calculated according to its descent (or ascent) profile. The travel time of each air segment fluctuates between the minimum and maximum possible values due to the potential for variations in aircraft speed. Moreover, arriving aircraft are expected to land on the runways at the target landing times, while departing aircraft are supposed to take off from the runways at the target take-off times.

In this context, the ASSP can be stated as follows: a set of aircraft arrive and leave within a TMA, with each following a pre-set landing or take-off air route. The goal is to determine a sequence and associated scheduled time at each waypoint for every aircraft by optimizing the given objective while subject to a variety of operational constraints. Without loss of generality, we outline the following assumptions to facilitate the discussion of our proposed ASSP: all waypoints are predetermined for each aircraft's landing or take-off procedure, all air traffic can be accommodated by the airport's terminal maneuvering airspace, operational failures (such as missed approaches, emergency landings or engine failures) are not considered relevant for this discussion, operating time is excluded from consideration, and only one aircraft is permitted to land or take-off on each runway at a time.

2.2 Problem formulation

The ASSP is formulated using a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ with a set of waypoints $\mathcal{V}$ and a set of arcs $\mathcal{A}$. The set of all aircraft is denoted by $\mathcal{F} = \mathcal{F}_{arr} \cup \mathcal{F}_{dep}$, where $\mathcal{F}_{arr}$ represents the set of arrival aircraft and $\mathcal{F}_{dep}$ represents the set of departure aircraft. Each aircraft $i \in \mathcal{F}$ follows a pre-defined air route for landing or take-off. The air route of aircraft i comprises a set of waypoints represented as $\mathcal{V}_i = (o, s_i, \dots, k, l, \dots, e_i, d)$. Here, waypoints o and d denote the dummy source and sink nodes, respectively, representing the first and last operations in the ASSP model. The waypoint s_i signifies the entry point for the arriving aircraft, and the runway for the departing ones. Conversely, e_i denotes the runway for arriving aircraft and the exit waypoint for those departing. An air segment within the aircraft's air route is defined by any two adjacent waypoints, noted as $(k, l) \in \mathcal{A}_i$. In this connection, $\mathcal{V}_i \subset \mathcal{V}$, $\mathcal{A}_i \subset \mathcal{A}$ in the digraph $\mathcal{G}$. The set of aircraft traversing waypoint k is denoted as $\mathcal{F}_k$.

We define the continuous decision variable t_{ik} to determine the time when aircraft i enters waypoint k , with $t_{ik} \geq 0$. We use the binary decision variable x_{ijk} to model the sequential relationship between aircraft i and j as they traverse the same waypoint k . Specifically, x_{ijk} is equal to 1 if aircraft i traverses waypoint k before aircraft j , and 0 otherwise. We impose $t_o = 0$ without loss of generality, and formulate the ASSP as follows:

$$(\text{ASSP}) : \min \quad z = \lambda \sum_{i \in \mathcal{F}} \rho_i + (1 - \lambda) \sum_{i \in \mathcal{F}} (t_{ie_i} - t_{is_i}) \quad (1a)$$

$$\text{s.t.} \quad t_{is_i} - t_o \geq \tau_{is_i}^{\text{early}} \quad \forall i \in \mathcal{F} \quad (1b)$$

$$t_{is_i} - t_o \leq \tau_{is_i}^{\text{late}} \quad \forall i \in \mathcal{F} \quad (1c)$$

$$t_{ik} - t_{il} \geq \underline{\theta}_{ik_{il}} \quad \forall i \in \mathcal{F}; \forall (k, l) \in \mathcal{A}_i \quad (1d)$$

$$t_{ik} - t_{il} \leq \bar{\theta}_{ik_{il}} \quad \forall i \in \mathcal{F}; \forall (k, l) \in \mathcal{A}_i \quad (1e)$$

$$t_{jk} - t_{ik} + M(1 - x_{ijk}) \geq \pi_{ijk} \quad \forall k \in \mathcal{V}; \forall i, j \in \mathcal{F}_k, i < j \quad (1f)$$

$$t_{ik} - t_{jk} + Mx_{ijk} \geq \pi_{jik} \quad \forall k \in \mathcal{V}; \forall i, j \in \mathcal{F}_k, i < j \quad (1g)$$

$$\rho_i \geq t_{ie_i} - \tau_{ie_i} \quad \forall i \in \mathcal{F}_{arr} \quad (1h)$$

$$\rho_i \geq t_{is_i} - \tau_{is_i}^{\text{target}} \quad \forall i \in \mathcal{F}_{dep} \quad (1i)$$

$$\rho_i \geq 0 \quad \forall i \in \mathcal{F} \quad (1j)$$

$$t_{ik} \geq 0 \quad \forall i \in \mathcal{F}; \forall k \in \mathcal{V}_i \quad (1k)$$

$$x_{ijk} \in \{0, 1\} \quad \forall k \in \mathcal{V}; \forall i, j \in \mathcal{F}_k \quad (1l)$$

where, the objective of our ASSP is to minimize a linear combination of total flight delay time and total dwell time. The dwell time refers to the duration that an aircraft spends from entering the TMA till landing at the runway (or from taking off until exiting the TMA for departing aircraft). The parameter $\lambda \in [0, 1]$ denotes the weight assigned by decision-makers (i.e., the ATCs) to express the relative importance of the total flight delay time with respect to the total dwell time. The variable ρ_i represents the flight delay time of aircraft i , while the dwell time of aircraft i is expressed as $t_{ie_i} - t_{is_i}$. Note that, the ASSP is identical to a total flight delay time minimization problem if $\lambda = 1$, while $\lambda = 0$ would lead to a total dwell time minimization problem.

Constraints (1b)–(1c) impose restrictions on each aircraft i by bounding its entry time s_i between the estimated earliest arrival time $\tau_{is_i}^{\text{early}}$ and the latest arrival time $\tau_{is_i}^{\text{late}}$. Constraints (1d)–(1e) cater to the variation of travel times in air segments (k, l) within a predefined range $[\underline{\theta}_{ik_{il}}, \bar{\theta}_{ik_{il}}]$. Constraints (1f)–(1g) represent separation

constraints, ensuring that if aircraft j follows aircraft i at the same waypoint k , the arrival time of aircraft j at waypoint k must not be less than the arrival time of aircraft i plus the minimum time-based interval π_{ijk} . Here, M is a sufficiently large number, such as the sum of maximum travel times of all aircraft across all air segments. Constraints (1h)–(1j) calculate the flight delay time for each aircraft. The flight delay time of arrival aircraft $i \in \mathcal{F}_{arr}$ is given by $\max\{0, t_{ie_i} - \tau_{ie_i}\}$, where $\tau_{ie_i} = \max\{\tau_{is_i}^{\text{early}} + \sum_{(k,l) \in \mathcal{A}_i} \theta_{ik-il} \tau_{ie_i}^{\text{target}}\}$, and $\tau_{ie_i}^{\text{target}}$ denotes the target landing time for aircraft i . The flight delay time of departure aircraft $i \in \mathcal{F}_{dep}$ is given by $\max\{0, t_{is_i} - \tau_{is_i}^{\text{target}}\}$, where $\tau_{is_i}^{\text{target}}$ denotes the target take-off time for aircraft i . Finally, Constraints (1k)–(1l) specify the domains of the decision variables.

3 The proposed solution method

The primary operations of the ASSP are sequencing and scheduling. Sequencing entails determining the sequence of aircraft at each waypoint (i.e., determining all values of the binary variable x_{ijk} , for $\forall k \in \mathcal{V}, \forall i, j \in \mathcal{F}_k$), and scheduling involves establishing the scheduled times for each aircraft at their respective waypoints (i.e., determining all values of the continuous variable t_{ik} , for $\forall i \in \mathcal{F}, \forall k \in \mathcal{V}_i$). It is important to note that when the aircraft sequence at each waypoint is defined, the ASSP converts into a linear programming, whereby the given sequence leads to the determination of the binary variable x_{ijk} . Consequently, given the sequence of aircraft at each waypoint, an optimal schedule, which includes the scheduled times for every aircraft at each waypoint, can be rapidly computed in polynomial time using available solvers such as Gurobi. Furthermore, when the aircraft sequence at each waypoint is manipulated, for example, by changing the positions of two aircraft, obtaining the optimal schedule for the new sequence is not difficult.

In this context, we develop a IFO algorithm for solving the ASSP. The idea underlying the proposed IFO algorithm is to create a subproblem of the ASSP by optimizing all continuous variables while dealing with just a select subset of binary variables. The remaining binary variables are temporarily fixed to values determined in previous iterations. In the subsequent sections, we will discuss the generation of an initial solution, outline the specific details of the IFO algorithm, and consider a RHC approach for speedup.

3.1 Generating an initial solution

We introduce a feasible sequence, indicated as S , to represent the sequence of operations for aircraft at the waypoints. Each aircraft $i \in \mathcal{F}$ prescribes to a landing (or take-off) route encompassing $|\mathcal{V}_i|$ waypoints, resulting in the sequence S including $2 \times \sum_i^{|\mathcal{F}|} |\mathcal{V}_i|$ elements in total. The first row of the sequence corresponds to the aircraft positions in the sequence, while the second row indicates the respective waypoints.

For example, consider a scenario with four aircraft, where aircraft a adheres to an air route $1 \rightarrow 2 \rightarrow 3$, aircraft b follows a air route $2 \rightarrow 3 \rightarrow 4$, aircraft c embarks on $1 \rightarrow 3 \rightarrow 4$ and finally, aircraft d navigates through $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$. A feasible sequence S could be denoted as $S = \begin{pmatrix} a & c & b & d & a & d & a & c & b & b & d & c & d \\ 1 & 1 & 2 & 1 & 2 & 2 & 3 & 3 & 3 & 4 & 3 & 4 & 4 \end{pmatrix}$. Here, the aircraft's positions captured in S represent the execution order of aircraft at each waypoint.

There are two essential propositions regarding the ASSP that play a critical role in generating the initial solution.

Proposition 1 *For each aircraft $i \in \mathcal{F}$ arriving at a given waypoint k along its landing or take-off air route, there exists both an earliest arrival time $t_{ik}^{\min} = \tau_{is_i}^{\text{early}} + \underline{\theta}_{is_i-ik}$ and a latest arrival time $t_{ik}^{\max} = \tau_{is_i}^{\text{early}} + \bar{\theta}_{is_i-ik}$. Here, $\underline{\theta}_{is_i-ik}$ and $\bar{\theta}_{is_i-ik}$ denote the minimum and maximum travel time for aircraft i from waypoint s_i to waypoint k , respectively.*

Proof Trivial. □

Proposition 2 *For any two consecutive aircraft $i \in \mathcal{F}$ and $j \in \mathcal{F}$ ($i \neq j$) traversing the same waypoint k , if $t_{ik}^{\max} < t_{jk}^{\min} + \pi_{ijk}$, then aircraft i will bypass waypoint k prior to aircraft j .*

Proof Trivial. □

Based on these propositions, the procedure to generate an initial solution for the ASSP is outlined in Algorithm 1. For each waypoint k , we evaluate all aircraft flying through the waypoint to determine one of the following three cases:

- Case 1: $t_{ik}^{\max} < t_{jk}^{\min} + \pi_{ijk}$
- Case 2: $t_{jk}^{\max} < t_{ik}^{\min} + \pi_{jik}$
- Case 3: All cases except case1 and case2.

In Case 1, aircraft i precedes aircraft j when flying through waypoint k . In Case 2, aircraft j precedes aircraft i when flying through waypoint k . In Case 3, aircraft i and j are sorted at the waypoints in non-decreasing order based on their target landing (or take-off) times. Once all aircraft are initially arranged in a feasible order at each waypoint, an initial sequence S is established. We then formulate a problem (ASSP_I), where all binary variables (x_{ijk} , for $\forall k \in \mathcal{V}, \forall i, j \in \mathcal{F}_k$) are fixed to values determined by the initial sequence S . We use the Gurobi solver to solve the problem (ASSP_I), obtaining its objective value $z(S)$. The default parameters of Gurobi are used except for the time limit, which may be adjusted as needed. The problem (ASSP_I) is formulated as follows:

$$(\text{ASSP}_I) : \min \quad z \quad (2a)$$

$$\text{s.t.} \quad (1b) - (1e), (1h) - (1k) \quad (2b)$$

$$t_{jk} \geq t_{ik} + \pi_{ijk} \quad \forall k \in \mathcal{V}; \forall i, j \in \mathcal{F}_k, i < j \quad (2c)$$

3.2 The IFO algorithm

We propose the IFO algorithm for the ASSP, detailed in Algorithm 2. The algorithm begins with an initial sequence S and its objective value $z(S)$, iteratively employing the fix-and-optimize procedure for improvements in the sequence. The fix-and-optimize procedure involves randomly selecting a sub-sequence S^R from the currently sequence S , regulated by two parameters: a center cc and a radius cr . Here, cc determines the centre position of S^R , while cr specifies the size of S^R . The sequencing constraints for aircraft in the S^R is allowed to be relaxed, enabling aircraft to change their order in the sequence. While, the aircraft in the remaining sequence, denoted as $S^{NR} = S \setminus S^R$, are not subject to changes in sequencing. We reduce the (ASSP) with a given S^R and S^{NR} to a problem (ASSP_R). Let $\mathcal{V}^{NR_{pre}}$, $\mathcal{V}^R$ and $\mathcal{V}^{NR_{suc}}$ denote the waypoint set of the preceding part in S^{NR} , the waypoint set in S^R and the waypoint set of the succeeding part in S^{NR} , respectively. Additionally, $\mathcal{F}^{NR_{pre}}$, $\mathcal{F}^R$ and $\mathcal{F}^{NR_{suc}}$ denote the sets of aircraft that traverse each waypoint k in $\mathcal{V}^{NR_{pre}}$, $\mathcal{V}^R$ and $\mathcal{V}^{NR_{suc}}$, respectively. The problem (ASSP_R) is subsequently formulated as follows:

Algorithm 1 The algorithm for generating the initial solution

```

1:  $S = \{\}$ 
2: for waypoint  $k$  in  $\mathcal{V}$  do
3:   Find all aircraft  $\mathcal{F}_k$  which are flying through the waypoint  $k$ .
4:   for aircraft  $i$  in  $\mathcal{F}_k$  do
5:     for aircraft  $j$  in  $\mathcal{F}_k$  do
6:       if  $i \neq j$  then
7:         if  $t_{ik}^{\max} < t_{jk}^{\min} + \pi_{ijk}$  then
8:           Add the order of aircraft  $i$  before aircraft  $j$  at waypoint  $k$  to  $S$ .
9:         else if  $t_{jk}^{\max} < t_{ik}^{\min} + \pi_{jik}$  then
10:          Add the order of aircraft  $j$  before aircraft  $i$  at waypoint  $k$  to  $S$ .
11:        else
12:          Sort aircraft  $i$  and  $j$  at waypoint  $k$  in non-decreasing order based on their
            target landing (or take-off) times, and add the order to  $S$ .
13:        end if
14:      end if
15:    end for
16:  end for
17: end for
18: Solve the problem (ASSPI) utilizing a commercial solver like Gurobi.
19: Obtain the initial sequence  $S$  and its objective value  $z(S)$ .
```

$$(\text{ASSP}_R) : \min \quad z \quad (3a)$$

$$\text{s.t.} \quad (1b) - (1e), (1h) - (1k) \quad (3b)$$

$$t_{jk} \geq t_{ik} + \pi_{ijk} \quad \forall k \in \mathcal{V}^{NR_{pre}} \cup \mathcal{V}^{NR_{suc}}; \forall i, j \in \mathcal{F}_k^{NR_{pre}} \cup \mathcal{F}_k^{NR_{suc}}, i < j \quad (3c)$$

$$t_{jk} \geq t_{ik} + \pi_{ijk} \quad \forall k \in \mathcal{V}^{NR_{pre}} \cap \mathcal{V}^R; \forall i \in \mathcal{F}_k^{NR_{pre}}, j \in \mathcal{F}_k^R \quad (3d)$$

$$t_{jk} \geq t_{ik} + \pi_{ijk} \quad \forall k \in \mathcal{V}^R \cap \mathcal{V}^{NR_{suc}}; \forall i \in \mathcal{F}_k^R, j \in \mathcal{F}_k^{NR_{suc}} \quad (3e)$$

$$t_{jk} - t_{ik} + M(1 - x_{ijk}) \geq \pi_{ijk} \quad \forall k \in \mathcal{V}^R; \forall i, j \in \mathcal{F}_k^R, i < j \quad (3f)$$

$$t_{ik} - t_{jk} + Mx_{ijk} \geq \pi_{jik} \quad \forall k \in \mathcal{V}^R; \forall i, j \in \mathcal{F}_k^R, i < j \quad (3g)$$

$$x_{ijk} \in \{0, 1\} \quad \forall k \in \mathcal{V}^R; \forall i, j \in \mathcal{F}_k^R \quad (3h)$$

In the (ASSP_R) formulation, we maintain the execution orders of all aircraft at each waypoint in the sequence S^{NR} unchanged (see constraints (3c)–(3e)). In contrast, we relax the sequencing constraints for all aircraft at their respective waypoints in the sequence S^R to allow for potential adjustments (see constraints (3f)–(3h)). We then apply a solver like Gurobi to solve the (ASSP_R), deriving a new sequence S' with its corresponding objective value $z(S')$. The objective value $z(S')$ is at least good as the currently best objective value $z(S)$. The default parameters of Gurobi are used except for the time limit, which may be adjusted as needed. Solving the (ASSP_R) would involve re-sequencing and re-scheduling for the aircraft in S^R , while only re-scheduling would occur for the aircraft in S^{NR} . This process may result in changes to the operation order of aircraft within the relaxed sequence S^R . We can easily re-encode a newly scheduled sequence by arranging the relaxed aircraft in a non-decreasing order based on their scheduled times at their respective waypoints. Since the aircraft order on S^{NR} stays fixed, no re-encoding is necessary for these aircraft.

For example, consider the sequence S discussed in Sect. 3.1. Given $cr = 8$ and $cc = 1$, the relaxed sub-sequence is $S^R = \begin{pmatrix} a & c & b \\ 3 & 3 & 3 \end{pmatrix}$. Relaxing the sequence constraints for these aircraft in S^R allows them to reorder and potentially improve the objective function value. Figure 1 shows a schematic diagram of this process. The disjunctive arcs, represented by dashed orange lines, indicate that these aircraft are subject to possible re-sequencing and re-scheduling operations. On the other hand, the remaining aircraft in the sequence follow conjunctive arcs represented by solid black lines. These aircraft are only subject to re-scheduling operations, meaning their sequencing remain unchanged.

Algorithm 2 The IFO algorithm for the ASSP

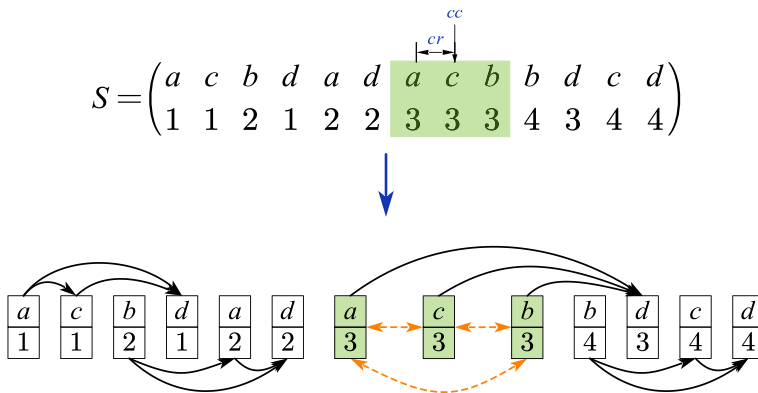


Fig. 1 A schematic diagram of the fix-and-optimize procedure

Input: Integers $iter_{\max}$, $noimproving_{\max}$, cr and $TL_{\max}$. A tabu list TL .

```

1:  $iter \leftarrow 1$ 
2:  $\xi \leftarrow 0$ 
3:  $TL \leftarrow \emptyset$ 
4:  $S, z(S) \leftarrow \text{Generate initial solution (see Algorithm 1)}$ 
5: while  $iter \leq iter_{\max}$  and  $\xi < noimproving_{\max}$  do
6:    $cc \leftarrow \text{Randomly select one from all positions of } S$ 
7:   if  $cc$  not in  $TL$  then
8:      $S', z(S') \leftarrow \text{The fix-and-optimize procedure}(S, cc, cr)$ 
9:     if  $z(S') < z(S)$  then
10:       $S \leftarrow S'$ 
11:       $z(S) \leftarrow z(S')$ 
12:       $\xi \leftarrow 0$ 
13:       $TabuMemoryFilter(TL)$ 
14:   else
15:      $TL \leftarrow TabuMemoryUpdate(TL, TL_{\max}, S', cc)$ 
16:      $\xi \leftarrow \xi + 1$ 
17:      $iter \leftarrow iter + 1$ 
18:   end if
19: end while
20: end while

```

Output: An improved sequence S and its objective value $z(S)$.

The standout feature of the fix-and-optimize procedure lies in the use of solvers for sequence improvement. Although optimization solvers usually require significant time to determine the optimal sequences and schedules for larger instances, they are highly efficient for smaller-sized instances. Therefore, by relaxing the sequencing constraints for a select few aircraft can effectively leverage the solver's capabilities and quickly yield an optimal or near-optimal solution for the relaxed aircraft. It should be note that our fix-and-optimize procedure is different with the sequence manipulation operators in works of [32, 33] and [34], all of which employed varied techniques such as recombination operators, swapping or insertion moves to obtain new (improved) sequences.

However, these sequence manipulations are often implemented randomly or myopically, neglecting the impact on the rest of the sequence. As a result, these sequence manipulations often fail to guide towards generating high-quality sequences.

During the search phase, a tabu search approach is utilized to avoid being trapped in local optima and revisiting the same solution. The tabu search strategy leverages an efficient memory structure to move away from previous solutions. We facilitate this process by introducing two operations: the tabu memory filter and the tabu memory update. The former operation eliminates all of the tabu moves from the current relaxation procedure (no aspiration criteria is used), while the latter operation updates the list of tabu moves during the search. We adopt a tabu list (TL) in the search phase that contains a maximum of TL_{max} moves. Each implemented move stores two key pieces of information: the current sequence S and the central parameter value cc . If an improved sequence S' with a lower objective value ($z(S') < z(S)$) is obtained, it replaces the current best sequence and objective value, and the search continues. If no improvement is identified, the count of no improvement increases, and the search goes on. The algorithm terminates based on two criteria: the maximum number of iterations ($iter_{max}$) and the maximum number of iterations without improvement ($noimproving_{max}$). Once the algorithm terminates, the improved sequence S and its objective value $z(S)$ are reported as the final solution.

3.3 Rolling horizon control approach

In this section, we present the RHC approach for expediting the process of deriving solutions. Given a defined time horizon $[H_{start}, H_{end}]$, any aircraft that meet the $\tau_{ie_i}^{early} \in [H_{start}, H_{end}]$ is subject to sequencing and scheduling. The RHC approach involves dividing the overall time horizon $[H_{start}, H_{end}]$ into $|W|$ operating time horizons each with the time length η . The time difference δ between two consecutive operating time horizons is referred to as the rolling period. Thus, the start and end times of the w th operating time horizon are denoted by $h_{start}^w = H_{start} + w\delta$ and $h_{end}^w = H_{start} + w\delta + \eta$, where $w = \{0, 1, \dots, |W|\}$. The original problem is then decomposed into a set of subproblems with respect to each operating time horizon. Within each operating time horizon, the IFO algorithm executes to solve the subproblem at hand, thereby generating sequencing and scheduling outcomes based on the currently available operational information.

In the RHC approach, each operating time horizon consists of two groups of aircraft: ongoing aircraft that have not completed scheduling by the end of the previous operating time horizon, and new aircraft arriving in the current operating time horizon awaiting scheduling. With respect to the current operating time horizon w , for each ongoing aircraft i , we can find a waypoint k in its air route that satisfies $t_{ik} \leq h_{start}^{w-1} + \delta$ and $t_{ik+1} > h_{start}^{w-1} + \delta$, where $k+1$ denotes the next waypoint after k . We then update the aircraft i to follow a air route from waypoint k to e_i with the $\tau_{ik}^{early} = \tau_{ik}^{late} = t_{ik}$, and optimize the aircraft i within time horizon w . The RHC approach is repeated until the end of the overall time horizon, and Algorithm 3 provides an overview of the RHC approach.

Algorithm 3 The RHC approach for the ASSP

```

1:  $w \leftarrow 0$ 
2:  $h_{start}^w \leftarrow H_{start}$ 
3:  $h_{end}^w \leftarrow h_{start}^w + \eta$ 
4:  $\mathcal{B}^{pre} \leftarrow \emptyset$  (Set of aircraft scheduled information in the previous operating time horizon)
5:  $\mathcal{B}^{current} \leftarrow$  Set of aircraft operational information with the  $\tau_{is_i}^{early} \in [h_{start}^w, h_{end}^w]$ 
6: while  $h_{start}^w < H_{end}$  do
7:   if  $\mathcal{B}^{pre}$  is not  $\emptyset$  then
8:     for each aircraft in  $\mathcal{F}^{pre}$  do
9:       Verify if aircraft  $i$  has finished its landing (or take-off) process in  $[h_{start}^{w-1}, h_{start}^{w-1} + \delta]$ .
10:      if aircraft  $i$  is not fully processed then
11:        Find a waypoint  $k$  (such that the start time of waypoint  $k$  is  $t_{ik} \leq h_{start}^{w-1} + \delta$ 
12:        and the start time of waypoint  $k + 1$  is  $t_{ik+1} > h_{start}^{w-1} + \delta$ ).
13:        Update aircraft  $i$  follow a air route from waypoint  $k$  to  $e_i$  with the  $\tau_{ik}^{early} =$ 
14:         $\tau_{ik}^{late} = t_{ik}$ .
15:        Add aircraft  $i$  scheduling status to  $\mathcal{B}^{current}$ .
16:      end if
17:    end for
18:  end if
19:  Solve the ASSP of set  $\mathcal{B}^{current}$  by the IFO algorithm (see Algorithm 2).
20:   $\mathcal{B}^{pre} \leftarrow \mathcal{B}^{current}$ 
21:   $w \leftarrow w + 1$ 
22:   $h_{start}^w \leftarrow h_{start}^{w-1} + \delta$ 
23:   $h_{end}^w \leftarrow h_{end}^{w-1} + \delta$ 
24:  Update  $\mathcal{B}^{current}$ 
25: end while

```

4 Experimental results

This section presents the numerical experiments conducted to evaluate the performance of the proposed the ASSP model and the solution method. Section 4.1 then details the schemes for generating test instances along with the guidelines for parameters setting. Subsequently, Sect. 4.2 presents an evaluation of our proposed solution method effectiveness, and Sect. 4.3 initiates a sensitivity analysis on the RHC configurations. Finally, Sect. 4.4 performs a solution sensitivity analysis with respect to the weight parameter and provides some insights.

All the models and algorithms are coded in Python programming language, utilizing Cython C extensions for faster code execution. The problems (ASSP), (ASSP_I) and (ASSP_R) are solved using the commercial solver Gurobi 10.0.1 [36] with default parameters, except for the time limit. All experiments are performed using Inter Core I7 1.10 GHz CPU and 16 G ARM in a Windows 10 operating environment.

4.1 Test instances and parameters setting

The test instances included in this study are derived from the real scheduling data of traffic flows within GBIA TMA in November 2019. There are three

classifications of air traffic conditions depending on the air traffic demand and passenger preferences, namely low-density traffic, normal-density traffic, and high-density traffic. We simulate 9 separate test instances with three different time horizons. For each time horizon, low-density, normal-density and high-density traffic were respectively generated and summarised in Table 1.

In all test instances, we only consider the north orientation approach for landing aircraft and do not include take-off air segments for departing aircraft. As outlined in the available aeronautical information publication, Fig. 2a shows the designed north orientation arrival routes used by landing aircraft to approach the hub. The GBIA features three parallel runways expressly built for arriving and departing procedures: runway 01/19, runway 02 L/20R, and runway 02R/20 L, all of which are shown in Fig. 2b. Among them, runway 01/19 handles both arrivals and departures, runway 02R/20 L is solely dedicated to arrivals, while runway 02 L/20R exclusively serves departures. The study assumes independent instrument approaches between runway 01/19 and runway 02R/20 L and segregated parallel approaches or departures between runway 02R/20 L and runway 02 L/20R.

The minimum time-based separation between two consecutive aircraft in the airspace is calculated based on the work of [37]. The minimum time-based separation matrices for operations on the same runway as well as those for close parallel runways are referenced from [38]. Table 2 summarizes the parameter values employed in our IFO algorithm. It classifies all test instances into three categories based on the number of aircraft: up to 40 aircraft, 40 to 100 aircraft, and over 100 aircraft, and these test instances within each group are solved using the given parameter values.

4.2 Computational comparison

In this section, we evaluate the performance of our IFO algorithm and the RHC approach based on the IFO algorithm (RHC+IFO). This evaluation involves a comparison against a variety of methods including:

- Gurobi: The commercial solver Gurobi.
- RHC+Gurobi: The RHC approach based on the Gurobi solver.
- BA: The bionomic algorithm proposed by [25].
- RHC+BA: The RHC approach based on the bionomic algorithm.
- VND: The variable neighborhood descent proposed by [32].
- RHC+VND: The RHC approach based on the variable neighborhood descent.
- ILS: The iterated local search proposed by [34].
- RHC+ILS: The RHC approach based on the iterated local search.

We re-implement BA, RHC+BA, VND, RHC+VND, ILS and RHC+ILS on the same computer used for executing our IFO algorithm and RHC+IFO. We have chosen to re-implement only BA, VND and ILS, instead of all the available algorithms

Table 1 The distributions for test instances

InstanceID	Time horizon	Aircraft number	Distribution of weight categories for arriving			Distribution of weight categories for departing			
			Arrival		Departure	Heavy		Light	
			Total	Medium		Heavy	Medium	Light	
1 L	1 h	13	8	5	1	6	1	4	0
1 N	1 h	62	30	32	6	24	6	26	0
1 H	1 h	72	39	33	7	32	6	27	0
2 L	2 h	25	15	10	2	12	1	8	1
2 N	2 h	126	64	62	12	52	12	50	0
2 H	2 h	144	78	66	15	63	13	53	0
3 L	3 h	37	22	15	4	17	2	12	1
3 N	3 h	187	92	95	18	74	19	76	0
3 H	3 h	215	115	100	23	92	20	80	0

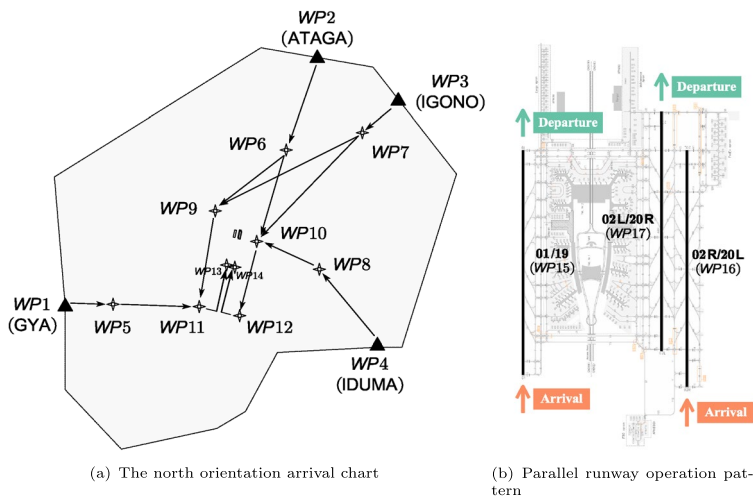


Fig. 2 GBA TMA configurations

Table 2 Values of the parameters for the IFO algorithm¹

Parameters	$ \mathcal{F} \leq 40$	$40 < \mathcal{F} \leq 100$	$ \mathcal{F} > 100$
$iter_{max}$	$\min \left\{ 10, \left\lfloor \frac{ \mathcal{F}_{arr} \times 5 + \mathcal{F}_{dep} }{4} \right\rfloor \right\}$	$\left\lfloor \frac{ \mathcal{F}_{arr} \times 5 + \mathcal{F}_{dep} }{4} \right\rfloor$	$\left\lfloor \frac{ \mathcal{F}_{arr} \times 5 + \mathcal{F}_{dep} }{4} \right\rfloor$
$noimproving_{max}$	3	10	10
cr	8	$\left\lfloor \frac{ \mathcal{F}_{arr} \times 5 + \mathcal{F}_{dep} }{12} \right\rfloor$	$\min \left\{ 25, \left\lfloor \frac{ \mathcal{F}_{arr} \times 5 + \mathcal{F}_{dep} }{20} \right\rfloor \right\}$
$timelimit_{initial}$ (s)	1	1	1
$timelimit_{search}$ (s)	1	2	3
TL_{max}	$\min \left\{ 10, \left\lfloor \frac{ \mathcal{F}_{arr} \times 5 + \mathcal{F}_{dep} }{4} \right\rfloor \right\}$	$\left\lfloor \frac{ \mathcal{F}_{arr} \times 5 + \mathcal{F}_{dep} }{4} \right\rfloor$	$\left\lfloor \frac{ \mathcal{F}_{arr} \times 5 + \mathcal{F}_{dep} }{4} \right\rfloor$

¹ $\lfloor x \rfloor$ is the greatest integer less than or equal to x ; $timelimit_{initial}$ and $timelimit_{search}$ denote the time limits set for the Gurobi solver to solve the problems (ASSP_I) and (ASSP_R), respectively

for solving the ASSP, as the BA is an algorithm that directly searching aircraft scheduled times at each waypoint instead of searching the aircraft sequences, and VND and ILS are conceptually similar to our proposed IFO algorithm. We initialize both VND and ILS using our proposed initial solutions generation algorithm to ensure fair comparisons. Moreover, we employ the Gurobi solver to determine the optimal aircraft scheduled times at each waypoint with predetermined aircraft sequences in VND and ILS.

We arbitrarily set $\lambda = 0.5$ and present the computational results in Table 3. The Gurobi is given a time limit of 3600 s to solve the (ASSP) within the overall time horizon. On the other hand, the RHC+Gurobi has a time limit of 60 s for each operating horizon w subproblem. For each test instance, the RHC parameters η and δ are determined based on Sect. 4.3. In Table 3, rows 2–4 list the outcomes of the Gurobi, including the best objective function value (z^*), CPU time (*Time*) and optimality

Table 3 The computational results on instances 1 L–3 H ($\lambda = 0.5$)

Index	1 L	1 N	1 H	2 L	2 N	2 H	3 L	3 N	3 H
Gurobi	z^* (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57781	73332.5
	Time (s)	0.01	2139.02	3600.16	0.09	3600.23	0.34	3600.28	3600.25
	Gap (%)	0	0.01	0.53	0	2.46	0	6.75	14.33
RHC+Gurobi	z^* (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57781	73414.5
	Time (s)	0.1	12.04	14.68	0.18	65.28	0.23	140.12	244.17
	z_{best} (s)	4592	18866	24096.5	8783	39886	12766	58060.5	75144
BA	z_{avg} (s)	4592	18916	24198.3	8797.2	39897.6	12800.1	58644.55	75625.5
	Time _{avg} (s)	62.13	160.26	184.12	91.2	359.22	101.04	833.25	23926.3
	z_{best} (s)	4592	18866	24083	8783	39861	12766	57944	74509
RHC+BA	z_{avg} (s)	4592	18867.3	24085.8	8783	39870.2	12766	58023.2	76727.04
	Time _{avg} (s)	38.02	92.43	126.72	57.9	225.32	69.08	395.23	660.35
	z_{best} (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57781	73299.5
VND	z_{avg} (s)	4592	18841.8	24017.1	8783	39743.65	12677.5	57821	73364
	Time _{avg} (s)	0.1	9.66	22.45	0.48	127.73	0.68	491.92	901.86
	z_{best} (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57781	73448.5
RHC+VND	z_{avg} (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57791.05	73448.9
	Time _{avg} (s)	0.1	7.51	12.74	0.3	77.81	0.47	167.5	287.01
	z_{best} (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57781	73269.5
ILS	z_{avg} (s)	4592	18840.15	24018.3	8783	39735.05	12677.5	57806.2	73308.05
	Time _{avg} (s)	0.1	10.68	18.11	0.48	116.24	0.74	402.93	697.59
	z_{best} (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57781	73448.5
RHC+ILS	z_{avg} (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57781	73448.5
	Time _{avg} (s)	0.1	8.21	14.06	0.32	64.28	0.48	114.1	262.43
	z_{best} (s)	4592	18838.5	24013.5	8783	39723.5	12677.5	57781	73448.5

Table 3 (continued)

Index	1 L	1 N	1 H	2 L	2 N	2 H	3 L	3 N	3 H
IFO	4592	18838.5	24013.5	8783	39723.5	49231.5	12677.5	57781	73230.5
	z_{best} (s)								
	4592	18840.15	24014.7	8783	39725.65	49239	12677.5	57787.95	73246.4
RHC+IFO	0.1	7.76	11.74	0.47	60.84	158.31	0.61	147.29	407.84
	$Time_{ang}$ (s)								
	4592	18838.5	24013.5	8783	39723.5	49267.5	12677.5	57781	73414.5
	4592	18838.5	24013.5	8783	39723.5	49267.5	12677.5	57781	73414.5
	z_{ang} (s)								
	0.1	5.26	9.9	0.3	45.75	77.69	0.42	68	180.88
	$Time_{ang}$ (s)								

gap (*Gap*) at the time limit. Rows 5–6 illustrate the results of RHC+Gurobi, reflecting the best objective function value (z^*) and CPU time (*Time*). The results from rows 7–30 are organized in blocks of three rows each for BA, RHC+BA, VND, RHC+VND, ILS, RHC+ILS, IFO, and RHC+IFO. These results include the best objective function value (z_{best}) in 10 runs for each test instance, the average objective function value (z_{avg}) in 10 runs for each instance, and the average CPU time ($Time_{avg}$).

The computational results indicate that our IFO algorithm significantly outperformed nine comparative methods in terms of solution quality. Specifically, for the instances 1 L, 1N, 1 H, 2 L, 2N, 3 L, and 3N, IFO algorithm achieved the solution quality equivalent to the best solution produced by Gurobi. For instances 2 H and 3 H, IFO algorithm excelled Gurobi by rendering superior solutions in both the best solution and the average solution. In comparison, BA and RHC+BA consistently generated low-quality solutions for all instances, excluding 1 L and 2 L. VND and ILS presented solutions with equivalent or superior quality compared to Gurobi's best solutions for all instances, yet our IFO algorithm exceeded their performance in instances 2 H and 3 H. Furthermore, solution qualities from RHC+IFO, RHC+Gurobi, RHC+VND, and RHC+ILS matched Gurobi's best output for instances 1 L, 1N, 1 H, 2 L, 2N, 3 L, and 3N. These algorithms, however, fell behind in instances 2 H and 3 H compared to the best solutions delivered by Gurobi.

Our RHC approach further accelerated the solving process by decomposing the original large problem into a series of smaller subproblems. The RHC+IFO algorithm enhanced the solving speed of the IFO algorithm by approximately 40% on average in terms of CPU time, while still delivering high-quality solutions. Moreover, MRHC+Gurobi, MRHC+BA, MRHC+VND, and MRHC+ILS also reduced CPU time compared to Gurobi, BA, VND, and ILS, respectively, without significantly compromising the solution quality. Although the IFO and RHC+IFO algorithms consumed slightly more CPU time than Gurobi for instances 1 L, 2 L, and 3 L, they use less CPU time than all eight compared methods for the other six instances. The reliability and effectiveness of our IFO and RHC+IFO algorithms are evident as their consistently delivery the best solutions across multiple runs, while maintaining reasonably short CPU times. These attributes designate our IFO and RHC+IFO algorithms as efficient and effective solutions for the ASSP.

4.3 Sensitivity analysis on RHC configurations

In this section, we investigate the optimization results obtained by RHC+IFO with respect to various configurations of RHC parameters η and δ . For the purpose of this demonstration, we arbitrarily select the results with respect to $\lambda = 0.5$. Tables 4, 5, 6 displays the values of z_{avg} , $Time_{avg}$ and $|W|$ for the nine instances 1 L–3 H, which are based on 10 distinct runs with different η and δ configurations for each instance. The best results, obtained under the optimal parameter combination, are highlighted in bold in Tables 4, 5, 6. These distinctive results are in alignment with the RHC+IFO results outlined in Table 3.

Table 4 The computational results for instances 1 L, 1N and 1 H under various RHC configurations ($\lambda = 0.5$)

η (s)	δ (s)	W	1 L		1N		1 H	
			z_{avg} (s)	$Time_{avg}$ (s)	z_{avg} (s)	$Time_{avg}$ (s)	z_{avg} (s)	$Time_{avg}$ (s)
900	300	10	4592	0.39	18945.5	7.1	24199	8.99
900	600	6	4592	0.15	18930.5	6	24201.5	5.5
900	900	4	4592	0.14	18882.5	6.1	24352	4.36
1800	600	4	4592	0.25	18838.5	8.47	24013.5	9.9
1800	1200	3	4592	0.18	18868.5	4.26	24050	8.34
1800	1800	2	4592	0.13	18858	4.98	24207	7.5
2700	900	2	4592	0.19	18838.5	7.85	24057.4	10.1
2700	1800	2	4592	0.1	18838.5	6.05	24068.4	6.4
2700	2700	2	4592	0.1	18838.5	5.26	24136.1	5.55

Table 5 The computational results for instances 2 L, 2N and 2 H under various RHC configurations ($\lambda = 0.5$)

η (s)	δ (s)	W	2 L		2N		2 H	
			z_{avg} (s)	$Time_{avg}$ (s)	z_{avg} (s)	$Time_{avg}$ (s)	z_{avg} (s)	$Time_{avg}$ (s)
900	300	22	8783	1.2	40242	24.38	49975.5	53.38
900	600	12	8783	0.66	40200	17.92	49692.05	54.19
900	900	8	8783	0.46	39985.5	15.92	49704.65	48.89
1800	600	10	8783	0.71	39723.5	64.53	49308.5	90.89
1800	1200	6	8783	0.38	39723.5	45.75	49308.5	87.11
1800	1800	4	8783	0.32	39761	51.23	49267.5	77.69
2700	900	6	8783	0.74	39791.5	26.69	49361	84.14
2700	1800	4	8783	0.45	39823.95	20.7	49336	90.91
2700	2700	3	8783	0.3	39801.5	39.7	49467.5	65.59
3600	1200	4	8783	0.9	39851.5	56.26	49354	60.78
3600	2400	3	8783	0.48	39897	50.52	49347.3	56.56
3600	3600	2	8783	0.3	39858.5	36.73	49650	56.4

A comparative analysis of the computational results of IFO algorithm (Table 3) and RHC+IFO algorithm (Tables 4, 5, 6) reveals that RHC+IFO algorithm consistently offers faster computational times across all instances compared to IFO algorithm. This is attributed to the RHC approach which decomposes the problem, thereby enabling quicker solutions for each subproblem with respect to the operation windows. The RHC approach, when applied with different parameter configurations, can lead to a variety of solutions. For instance, in dealing with low-density traffic instances (i.e., instances 1 L, 2 L, and 3 L), the longest time horizon yields the best results evidenced by the shortest CPU time. This is possible as the reduction in subdivisions leads to a decrease in CPU time for these instances. On the other hand, for

Table 6 The computational results for instances 3 L, 3N and 3 H under various RHC configurations ($\lambda = 0.5$)

η (s)	δ (s)	W	3 L		3N		3 H	
			z_{avg} (s)	$Time_{avg}$ (s)	z_{avg} (s)	$Time_{avg}$ (s)	z_{avg} (s)	$Time_{avg}$ (s)
900	300	34	12677.5	1.95	58406.5	47.08	74442.5	93.16
900	600	18	12677.5	1.09	58461	34.11	74342	74.98
900	900	12	12677.5	0.65	58231.5	39.88	74451.5	82.5
1800	600	16	12677.5	1.24	57825.5	84.81	73592.5	167.69
1800	1200	9	12677.5	0.72	57781	68	73414.5	180.88
1800	1800	6	12677.5	0.44	57902.5	58.62	73857.9	182.21
2700	900	10	12677.5	1.26	57894.25	50.12	73669.2	150.64
2700	1800	6	12677.5	0.65	57940	43.26	73525.7	167.9
2700	2700	4	12677.5	0.48	57868	32.75	73747.45	140.22
3600	1200	7	12677.5	1.48	57952.5	71.03	73637	121.77
3600	2400	4	12677.5	0.81	58073.5	67.64	73510.5	98.88
3600	3600	3	12677.5	0.42	58019	52.09	74258.5	113.14

normal-density and high-density traffic instances (i.e., instances 1N, 1 H, 2N, 2 H, 3N, and 3 H), a time horizon of 1800s garners optimal results. Exceptionally short time horizons lead to over subdivision of the original problem and thus the solution quality is far worse compared to the optimal solution. Conversely, exceedingly large time horizons decrease the possibility of obtaining optimal solutions for the sub-problems and consequently lead to slightly lower quality solutions.

It is worth noting that, the RHC approach provides enhanced operational flexibility while solving ASSP. Even though we expect our model to provide a static solution for a particular set of aircraft, the inherent flexibility of the proposed RHC approach allows it to also accommodate a dynamic environment. As time progresses, some aircraft complete their landing or take-off process while the others are ongoing, and more aircraft are waiting to be scheduled in the TMA. The RHC method could be applied to control the roll dynamics, which involves rerunning the model and scheduling the take-off or landing process for the new set of aircraft satisfying boundary conditions.

4.4 Solution sensitivity to the weight parameter

In this section, we present our numerical results obtained from the IFO algorithm by varying values of the weight parameter λ . We systematically increment the value of λ from 0 to 1, with a step size of 0.1. Figure 3 illustrates the effect of varying the weight parameter λ on the total flight time and total dwell time for instances 1 L–3 H.

Upon analyzing Fig. 3, it's clear that as the weight parameter λ increases, the importance of total flight delay time in the ASSP model correspondingly increases, leading to solutions with smaller total flight delay time, while the total dwell time gradually increase. Notably, when λ takes the extreme values of 0 or 1 (representing

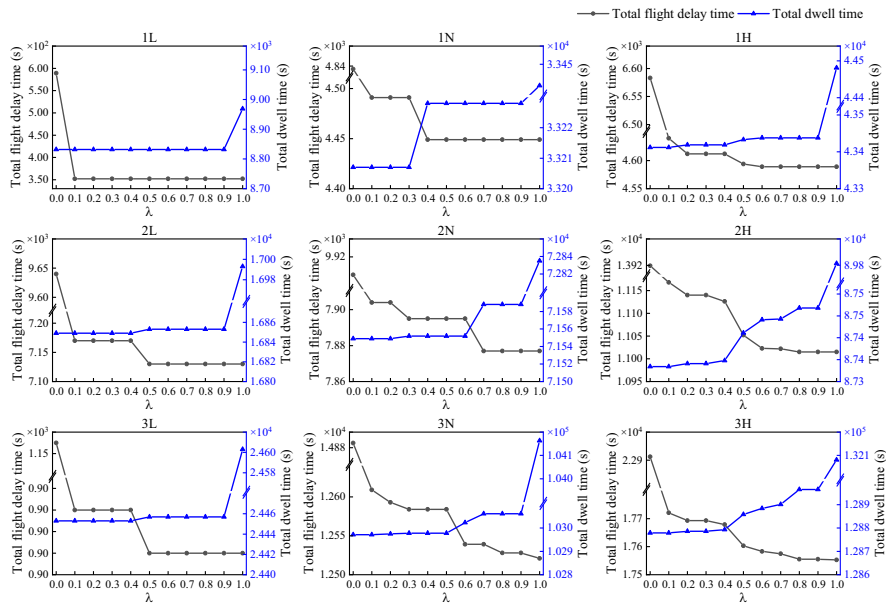


Fig. 3 Effect on the total flight delay time and total dwell time of weight parameter λ

the single objective of minimizing either total flight delay time or total dwell time), we observe a sharp increase in the amount of flight delay time or dwell time. The results imply that during the operations of aircraft landing and take-off within the TMA, minimizing total flight delay time and reducing total dwell time are not mutually exclusive objectives. Indeed, flight delay time reduction typically results from the decline in aircraft dwell time. If we focus solely on one objective and exclude the other during aircraft landing and take-off operations, however, we may end up with solutions that yield undesirable performance. Therefore, it is recommended to incorporate both objectives into our approach. These detailed trade-off information can assist ATCs in determining the optimal compromise between different performance indices using ASSP model.

We further compare the performance of our IFO algorithm against those of Gurobi, VND and ILS with varying values of the weight parameter λ . In the case of Gurobi, a time limit of 3600 s was set. Our IFO algorithm, as well as VND and ILS, yields the same solution quality as Gurobi's best for instances 1 L, 1N, 1 H, 2 L, 2N, 3 L, and 3N. While, for instances 2 H and 3 H, these algorithms exceed the performance of Gurobi's best solutions. Figures 4 and 5 depict the percentage improvement in the best solution and CPU time achieved by our IFO algorithm, VND, and ILS, compared to Gurobi. The percentage improvement in the best solution and CPU time were calculated as $(z^* - z_{best})/z_{best} \times 100$ and $(Time^* - Time_{best})/Time_{best} \times 100$, respectively. Here, z^* and $Time^*$ represent the best solution and CPU time for Gurobi, while z_{best} and $Time_{best}$ refer to the best solution and CPU time given by IFO algorithm, VND or ILS. As per the results, our IFO

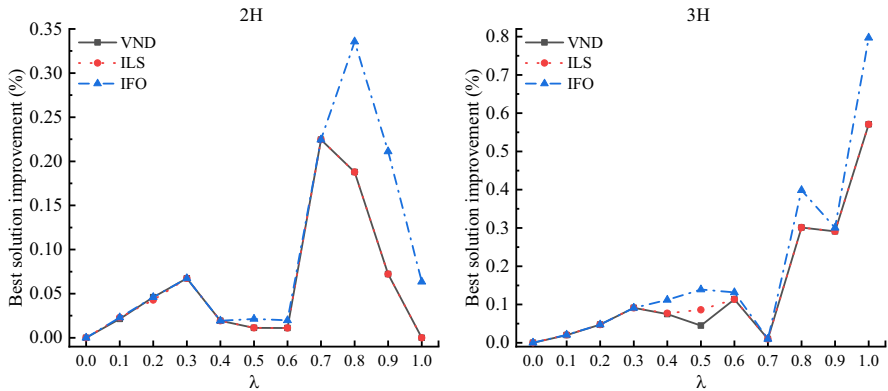


Fig. 4 Best solution improvement obtained by using VND, ILS and IFO algorithms compare to Gurobi in instances 2 H and 3 H

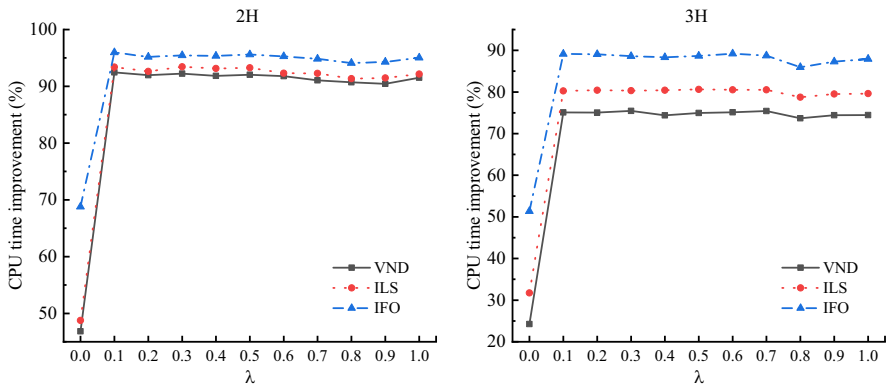


Fig. 5 CPU time improvement obtained by using VND, ILS and IFO algorithms compare to Gurobi in instances 2 H and 3 H

algorithm outperforms VND, ILS, and Gurobi in delivering both the best solution quality and CPU time, regardless of the value of λ .

5 Conclusion

In this paper, we develop an efficient IFO algorithm for solving the ASSP. The core of this algorithm lies in an iterative process of deconstruction and reconstruction (fix-and-optimize) of an aircraft sequence. To enhance the selection of such sequences, we incorporate the tabu search within the search phase to steer the process towards the more high-priced sequence. Moreover, we employ the RHC strategy to expedite the solution derivation process, realized by decomposing the

original problem into a set of small-scale subproblems. To evaluate the performance of our IFO algorithm, we conduct comprehensive computational experiments with real traffic data from GBIA. We further compare the performance of our IFO algorithm and three state-of-the-art solution algorithms for the ASSP, and also the solver Gurobi. The computational results demonstrate the effectiveness of our IFO algorithm in delivering high-quality solutions, and the benefits of the RHC approach in computational efficiency.

For future research work, there is potential for the development of an exact solution method to address large ASSP instances. Additionally, exploring the adaptability of our IFO algorithm for other challenging combinatorial optimization problems could be a promising avenue.

Acknowledgements This work was supported in part by the China Postdoctoral Science Foundation [Grant 2022T150319 and 2022M711611] and the Humanities and Social Science Fund of Ministry of Education [Grant 23YJC790027].

Availability of data and materials The data that support the findings of this study are not openly available due to reasons of sensitivity and are available from the corresponding author upon reasonable request.

References

1. ACI: Latest air travel outlook reveals 2024 to be a milestone for global passenger traffic. Report, Airports Council International (2023). <https://aci.aero/2023/09/27/latest-air-travel-outlook-reveals-2024-to-be-a-milestone-for-global-passenger-traffic>
2. Boysen, N., Friedner, M.: Scheduling aircraft landings to balance workload of ground staff. *Comput. Ind. Eng.* **60**(2), 206–217 (2011). <https://doi.org/10.1016/j.cie.2010.11.002>
3. Liu, W., Delahaye, D., Zhao, Q., Notry, P.: Benefit of wind networking for aircraft arrival scheduling in terminal manoeuvring area. *Comput. Ind. Eng.* **182**, 109418 (2023). <https://doi.org/10.1016/j.cie.2023.109418>
4. Balakrishnan, H., Chandran, B.G.: Scheduling aircraft landings under constrained position shifting. In: AIAA Guidance, Navigation, and Control Conference and Exhibit, p. 6320. Keystone, USA (2006)
5. Prakash, R., Piplani, R., Desai, J.: An optimal data-splitting algorithm for aircraft scheduling on a single runway to maximize throughput. *Transp. Res. Part C Emerg. Technol.* **95**, 570–581 (2018). <https://doi.org/10.1016/j.trc.2018.07.031>
6. Ball, M., Barnhart, C., Nemhauser, G., Odoni, A.: Air transportation: irregular operations and control. *Handb. Oper. Res. Manag. Sci.* **14**, 1–67 (2007). [https://doi.org/10.1016/S0927-0507\(06\)14001-3](https://doi.org/10.1016/S0927-0507(06)14001-3)
7. Bennell, J.A., Mesgarpour, M., Potts, C.N.: Airport runway scheduling. *4OR* **9**(2), 115–138 (2011). <https://doi.org/10.1007/s10288-011-0172-x>
8. Bennell, J.A., Mesgarpour, M., Potts, C.N.: Airport runway scheduling. *Ann. Oper. Res.* **204**(1), 249–270 (2013). <https://doi.org/10.1007/s10479-012-1268-1>
9. Ng, K.K.H., Lee, C.K.M., Chan, F.T.S., Lv, Y.: Review on meta-heuristics approaches for airside operation research. *Appl. Soft Comput.* **66**, 104–133 (2018). <https://doi.org/10.1016/j.asoc.2018.02.013>
10. Ikli, S., Mancel, C., Mongeau, M., Olive, X., Rachelson, E.: The aircraft runway scheduling problem: a survey. *Comput. Oper. Res.* (2021). <https://doi.org/10.1016/j.cor.2021.105336>
11. Beasley, J.E., Krishnamoorthy, M., Sharaiha, Y.M., Abramson, D.: Scheduling aircraft landings-the static case. *Transp. Sci.* **34**(2), 180–197 (2000). <https://doi.org/10.1287/trsc.34.2.180.12302>
12. Bianco, L., Dell’Olmo, P., Giordani, S.: Scheduling models for air traffic control in terminal areas. *J. Sched.* **9**(3), 223–253 (2006). <https://doi.org/10.1007/s10951-006-6779-7>

13. Mascis, A., Pacciarelli, D.: Job-shop scheduling with blocking and no-wait constraints. *Eur. J. Oper. Res.* **143**, 498–517 (2002). [https://doi.org/10.1016/S0377-2217\(01\)00338-1](https://doi.org/10.1016/S0377-2217(01)00338-1)
14. D'Ariano, A., D'Urgolo, P., Pacciarelli, D., Pranzo, M.: Optimal sequencing of aircrafts take-off and landing at a busy airport. In: 13th International IEEE Conference on Intelligent Transportation Systems, pp. 1569–1574 (2010). <https://doi.org/10.1109/ITSC.2010.5625114>
15. D'Ariano, A., Pacciarelli, D., Pistelli, M., Pranzo, M.: Real-time scheduling of aircraft arrivals and departures in a terminal maneuvering area. *Networks* **65**(3), 212–227 (2015). <https://doi.org/10.1002/net.21599>
16. Samà, M., D'Ariano, A., Pacciarelli, D.: Rolling horizon approach for aircraft scheduling in the terminal control area of busy airports. *Transp. Res. Part E Logist. Transp. Rev.* **60**, 140–155 (2013). <https://doi.org/10.1016/j.tre.2013.05.006>
17. Samà, M., D'Ariano, A., D'Ariano, P., Pacciarelli, D.: Optimal aircraft scheduling and routing at a terminal control area during disturbances. *Transp. Res. Part C Emerg. Technol.* **47**, 61–85 (2014). <https://doi.org/10.1016/j.trc.2014.08.005>
18. Samà, M., D'Ariano, A., Corman, F., Pacciarelli, D.: Metaheuristics for efficient aircraft scheduling and re-routing at busy terminal control areas. *Transp. Res. Part C Emerg. Technol.* **80**, 485–511 (2017). <https://doi.org/10.1016/j.trc.2016.08.012>
19. Samà, M., D'Ariano, A., D'Ariano, P., Pacciarelli, D.: Scheduling models for optimal aircraft traffic control at busy airports: tardiness, priorities, equity and violations considerations. *Omega* **67**, 81–98 (2017). <https://doi.org/10.1016/j.omega.2016.04.003>
20. Gui, D., Le, M., Huang, Z., Zhang, J., D'Ariano, A.: Optimal aircraft arrival scheduling with continuous descent operations in busy terminal maneuvering areas. *J. Air Transp. Manag.* (2023). <https://doi.org/10.1016/j.jairtraman.2022.102344>
21. Faye, A.: Solving the aircraft landing problem with time discretization approach. *Eur. J. Oper. Res.* **242**(3), 1028–1038 (2015). <https://doi.org/10.1016/j.ejor.2014.10.064>
22. Beasley, J.E., Sonander, J., Havelock, P.: Scheduling aircraft landings at London heathrow using a population heuristic. *J. Oper. Res. Soc.* **52**(5), 483–493 (2001). <https://doi.org/10.1057/palgrave.jors.2601129>
23. Hu, X.B., Chen, W.H.: Receding horizon control for aircraft arrival sequencing and scheduling. *IEEE Trans. Intell. Transp. Syst.* **6**(2), 189–197 (2005). <https://doi.org/10.1109/tits.2005.848365>
24. Hu, X.-B., Di Paolo, E.: A ripple-spreading genetic algorithm for the aircraft sequencing problem. *Comput. Oper. Res.* **19**(1), 77–106 (2011). https://doi.org/10.1162/EVCO_a_00011
25. Pinol, H., Beasley, J.E.: Scatter search and bionomic algorithms for the aircraft landing problem. *Eur. J. Oper. Res.* **171**(2), 439–462 (2006). <https://doi.org/10.1016/j.ejor.2004.09.040>
26. Atkin, J.A.D., Burke, E.K., Greenwood, J.S., Reeson, D.: Hybrid metaheuristics to aid runway scheduling at London heathrow airport. *Transp. Sci.* **41**(1), 90–106 (2007). <https://doi.org/10.1287/trsc.1060.0163>
27. Furini, F., Kidd, M.P., Persiani, C.A., Toth, P.: Improved rolling horizon approaches to the aircraft sequencing problem. *J. Sched.* **18**(5), 435–447 (2015). <https://doi.org/10.1007/s10951-014-0415-8>
28. Zhi-Hui, Z., Jun, Z., Yun, L., Ou, L., Kwok, S.K., Ip, W.H., Kaynak, O.: An efficient ant colony system based on receding horizon control for the aircraft arrival sequencing and scheduling problem. *IEEE Trans. Intell. Transp. Syst.* **11**(2), 399–412 (2010). <https://doi.org/10.1109/tits.2010.2044793>
29. Xu, B.: An efficient ant colony algorithm based on wake-vortex modeling method for aircraft scheduling problem. *J. Comput. Appl. Math.* **317**, 157–170 (2017). <https://doi.org/10.1016/j.cam.2016.11.043>
30. Hancerliogullari, G., Rabadi, G., Al-Salem, A.H., Kharbeche, M.: Greedy algorithms and metaheuristics for a multiple runway combined arrival-departure aircraft sequencing problem. *J. Air Transp. Manag.* **32**, 39–48 (2013). <https://doi.org/10.1016/j.jairtraman.2013.06.001>
31. Hammouri, A.I., Braik, M.S., Al-Betar, M.A., Awadallah, M.A.: Isa: a hybridization between iterated local search and simulated annealing for multiple-runway aircraft landing problem. *Neural Comput. Appl.* **32**(15), 11745–11765 (2019). <https://doi.org/10.1007/s00521-019-04659-y>
32. Salehipour, A., Moslemi Naeni, L., Kazemipour, H.: Scheduling aircraft landings by applying a variable neighborhood descent algorithm: runway-dependent landing time case. *Int. J. Appl. Oper. Res.* **1**, 39–49 (2009)
33. Salehipour, A., Modarres, M., Moslemi Naeni, L.: An efficient hybrid meta-heuristic for aircraft landing problem. *Comput. Oper. Res.* **40**(1), 207–213 (2013). <https://doi.org/10.1016/j.cor.2012.06.004>

34. Sabar, N.R., Kendall, G.: An iterated local search with multiple perturbation operators and time varying perturbation strength for the aircraft landing problem. *Omega* **56**, 88–98 (2015). <https://doi.org/10.1016/j.omega.2015.03.007>
35. Ahmadian, M.M., Salehipour, A.: Heuristics for flights arrival scheduling at airports. *Int. Trans. Oper. Res.* **29**(4), 2316–2345 (2020). <https://doi.org/10.1111/itor.12901>
36. Gurobi Optimization, L.: Gurobi optimizer reference manual (2022)
37. Huo, Y., Delahaye, D., Sbihi, M.: A dynamic control method for extended arrival management using enroute speed adjustment and route change strategy. *Transp. Res. Part C Emerg. Technol.* (2023). <https://doi.org/10.1016/j.trc.2023.104064>
38. Solak, S., Sölveling, G., Clarke, J.-P.B., Johnson, E.L.: Stochastic runway scheduling. *Transp. Sci.* **52**(4), 917–940 (2018). <https://doi.org/10.1287/trsc.2017.0784>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Dongdong Gui¹ · Meilong Le¹ · Xiaodong Luo³ · Zhouchun Huang² 

✉ Meilong Le
mlle@nuaa.edu.cn

✉ Zhouchun Huang
zhouchun.huang@nuaa.edu.cn

Dongdong Gui
guidongdong@nuaa.edu.cn

Xiaodong Luo
xiaodongluo@cuhk.edu.cn

¹ College of Civil Aviation, Nanjing University of Aeronautics and Astronautics, Nanjing, China

² College of Economics and Management, Nanjing University of Aeronautics and Astronautics, Nanjing, China

³ Shenzhen Research Institute of Big Data, Shenzhen, China